

2. Using C++ Arrays, Loops, and Math Functions

2.1	Motivating application: computing vorticity	16
2.2	Declaring constant variables	18
2.3	For-loops	18
2.4	The scope of variables in for-loops.	19
2.5	C-style arrays	20
2.6	The <code>cmath</code> header	22
2.7	Homework	23

Introduction

This chapter introduces the basic C++ concepts required to create a grid of points, compute and store a function at each point, and then approximate the derivative of the function using finite differences. For that, we focus on declaring constants, using for-loops to perform repetitive tasks, storing data in simple “C-style” arrays, and accessing common math operations through the `cmath` header file. The end-of-chapter homework applies all of these concepts to approximate the vorticity within a laminar boundary layer using finite differences.

2.1 Motivating application: computing vorticity

When analyzing the results of a CFD simulation, engineers routinely examine the *vorticity* field $\boldsymbol{\omega}$, defined as the curl of the velocity field,

$$\boldsymbol{\omega} = \nabla \times \mathbf{u}. \quad (2.1)$$

The vorticity field can be less physically intuitive than the velocity field, and it may not be immediately clear why it is important. Here, we highlight a few of its characteristics. First, because it is the curl of a vector field ($\mathbf{u}$), the vorticity field is itself a vector field,

$$\boldsymbol{\omega}(\mathbf{x}, t) = \omega_x(\mathbf{x}, t) \mathbf{i} + \omega_y(\mathbf{x}, t) \mathbf{j} + \omega_z(\mathbf{x}, t) \mathbf{k}. \quad (2.2)$$

Second, vorticity is often described as a measure of local “rotation” within a fluid flow. To understand why, consider the small fluid parcel illustrated as a grey cube in Fig. 2.1. As the parcel travels along its pathline, shown as a dashed curve, it simultaneously rotates about its center of mass with angular velocity $\boldsymbol{\Omega}$. Extending this idea to the full flow field, we can define an angular velocity field $\boldsymbol{\Omega}(\mathbf{x}, t)$ that describes the local rotation of fluid parcels throughout the domain. It can then be shown that $\boldsymbol{\omega}$ is simply twice this angular velocity field,

$$\boldsymbol{\omega} = 2\boldsymbol{\Omega}(\mathbf{x}, t). \quad (2.3)$$

The above kinematic interpretation of vorticity, in terms of $\boldsymbol{\Omega}$, is useful, but does not explain why vorticity is of practical importance. Here we mention three reasons:

- i. Vorticity is generated by viscous effects. Consequently, examining the vorticity field helps engineers understand where and how viscosity impacts a fluid flow.
- ii. Vorticity is particularly important in turbulent flows, where it plays a key role in “*vortex stretching*” and the “*energy cascade*.”
- iii. Vorticity is useful for visualizing fluid flows. As a CFD engineer, you may be asked to generate and interpret *vortex-lines* and isosurfaces of the vorticity field.

In practice, CFD codes typically solve for $\mathbf{u}$ and p , and vorticity is computed in post-processing as a derived quantity. For demonstration, this chapter considers the steady, 2D, flat-plate boundary layer sketched in Fig. 2.2. In 2D flows, the x and y components of vorticity vanish ($\omega_x = \omega_y = 0$), and the vorticity can be interpreted as a scalar field,

$$\omega = \omega_z = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y}. \quad (2.4)$$

Furthermore, in flat-plate boundary layers flowing in the downstream x -direction, the gradient $\partial v / \partial y$ is usually small,

$$\frac{\partial v}{\partial x} \ll \frac{\partial u}{\partial y}, \quad (2.5)$$

and the vorticity can be approximated as

$$\omega \approx -\frac{\partial u}{\partial y}. \quad (2.6)$$

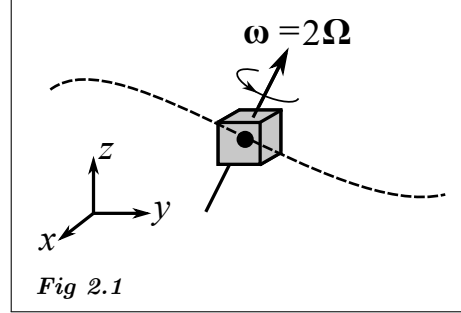
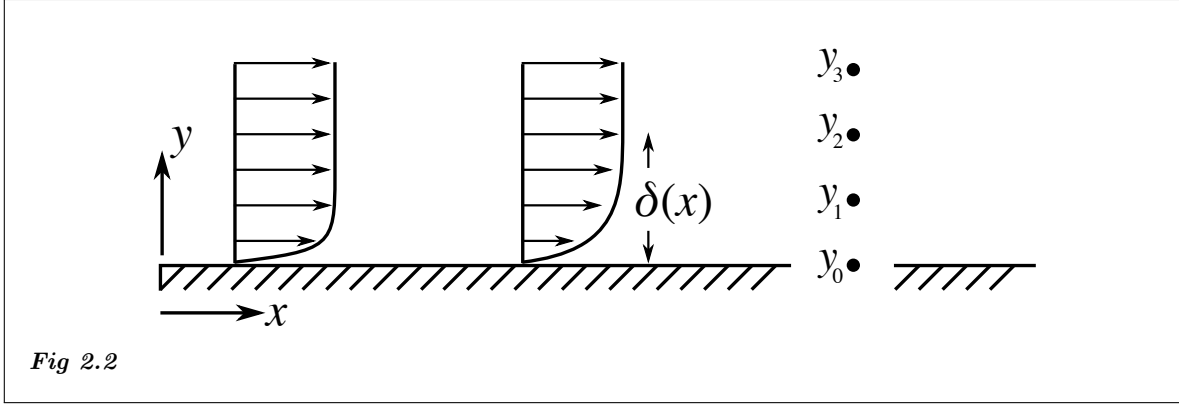


Fig 2.1



Let's now suppose that a CFD simulation outputs the downstream velocity $u(x, y)$ at a fixed downstream location x , sampled at N equispaced points in the wall-normal direction,

$$y_j = jh, \quad h = \frac{\delta(x)}{N-1}, \quad j = 0, 1, \dots, N-1, \quad (2.7)$$

where $\delta(x)$ is the boundary layer thickness. Figure 2.2 shows the case where $N = 4$. Note that $y_0 = 0$ and $y_{N-1} = \delta(x)$. In this chapter's homework, we write a C++ program that uses finite differences to approximate $\partial u / \partial y$ at each of the N grid points, and generate a plot of ω as a function of distance from the plate. Rather than using data from an actual CFD simulation, we use the following analytical approximation for the Blasius boundary layer,¹

$$u(x, y) = U_\infty \sin \left[\frac{\pi y}{2\delta(x)} \right], \quad \delta(x) = 4.795 \sqrt{\frac{\nu x}{U_\infty}}, \quad (2.8)$$

where U_∞ is the constant free-stream velocity, and ν is the fluid's kinematic viscosity. On the interior grid points ($1 \leq j \leq N-2$), we will approximate $\partial u / \partial y$ using the centered difference formula

$$\frac{\partial u_j}{\partial y} \approx \frac{u_{j+1} - u_{j-1}}{2h}, \quad (2.9)$$

where u_j denotes the measured velocity at location y_j . On the first and last grid point, we will use the following one-sided finite difference formulas,

$$\frac{\partial u_0}{\partial y} \approx \frac{-3u_0 + 4u_1 - u_2}{2h}, \quad (2.10)$$

$$\frac{\partial u_{N-1}}{\partial y} \approx \frac{3u_{N-1} - 4u_{N-2} + u_{N-3}}{2h}, \quad (2.11)$$

¹This approximation uses the Kármán-Pohlhausen momentum integral method with an assumed sine velocity profile.

2.2 Declaring constant variables

In C++, the keyword `const` can be used to declare a variable whose value cannot be changed after it is initialized. For example, we can define the variable `pi` using the syntax below:

```
const double pi = 3.141592653589793;
```

Note that a `const` variable must be assigned a value at the same time it is declared. For example, the following is invalid, and will generate a compiler error:

```
const double pi;  
pi = 3.141592653589793;
```

Using `const` is good practice for values that are meant to be fixed, such as physical constants.

2.3 For-loops

A *for-loop* repeatedly executes a code block a fixed number of times. This is useful for repetitive tasks, such as time stepping or operating on arrays. File (2.1) demonstrates a basic for-loop that computes and displays 7 grid points y_j when $\delta = 5$.

File 2.1: A simple for-loop.

```
1 #include <iostream>  
2  
3 int main()  
4 {  
5     // Declare variables  
6     int N = 7;  
7     double delta = 5.0;  
8     double h = delta/(N-1);  
9  
10    // compute and display grid points y  
11    for (int j = 0; j < N; j++ )  
12    {  
13        std::cout << "j = "<< j << ", y = " << j*h << std::endl;  
14    }  
15  
16    return 0;  
17 }
```

Lines 11–14 of File (2.1) demonstrate one classic approach of coding a C++ for-loop that performs N iterations.

```
11 for (int j = 0; j < N; j++)  
12 {  
13     ...  
14 }
```

Let's focus on the syntax on line 11. Following the keyword `for`, there are three statements inside the parentheses:

1. The first statement, `int j = 0`, initializes an integer variable `j`, often called the *loop index*, to zero.
2. The second statement, `j < N`, sets the condition for when the loop terminates.
3. The third statement, `j++`, increments the index by 1 after each iteration, using the increment operator introduced in Table 1.1.

Together, these three statements perform `N` iterations by increasing `j` by one each time, until `j < N` becomes false. For each iteration `j = 0, 1, ..., N-1`, a for-loop executes the code block contained in the subsequent braces `{ }`. In the case of File (2.1), this performs the single statement:

```
13 std::cout << "j = " << j << ", y = " << j*h << std::endl;
```

The statement above is worth studying because it demonstrates two new concepts. First, it shows that we can perform a mathematical operation (in this case, `j*h`) directly within a `std::cout` statement. Second, it shows that the loop index `j` can be used in computations. Compiling and running File (2.1) produces the output below:

```
j = 0, y = 0
j = 1, y = 0.833333
j = 2, y = 1.66667
j = 3, y = 2.5
j = 4, y = 3.33333
j = 5, y = 4.16667
j = 6, y = 5
```

2.4 The scope of variables in for-loops.

It is important to consider the scope of variables defined in for-loops. Consider the example in File (2.2) below.

File 2.2: A demonstration of variable scope in for-loops

```
1 for (int j = 0; j < N; j++ )
2 {
3     double y = j*h; // declare y in loop
4     std::cout << "j = " << j << ", y = " << y << std::endl;
5 }
```

In this example, the variable `y` is declared inside the loop block. Consequently, a new `y` is created at the start of each iteration and destroyed at the end of that iteration. Its value does not persist between iterations, and it cannot be accessed outside the loop braces `{ }`. For demonstration, consider the for-loop in File (2.3) below.

File 2.3: Variables defined in the block of a for-loop are destroyed at the end of each iteration.

```
1 #include <iostream>
2
3 int main()
4 {
5     for (int j = 0; j < 3; j++)
```

```

6  {
7      double y = 0.0;    // declare y inside loop
8      y = y + 1.0;       // increment y
9      std::cout << "Iteration " << j << ", y = " << y << std::endl;
10 }
11 return 0;
12 }

```

If `y` persisted across iterations, we would expect its value to increment by one each iteration. Instead, File (2.3) generates the output below:

```

Iteration 0, y = 1
Iteration 1, y = 1
Iteration 2, y = 1

```

Nevertheless, declaring variables inside a for-loop block is common practice in C++. In many cases, it is considered best-practice, because it keeps the variable local to the loop and avoids its unintended use elsewhere. Just keep in mind that such variables are reset after every iteration.

For-loops introduce the additional question of what is the scope of the loop index. In File (2.2), the loop index `j` is declared within the `for` statement itself:

```
for (int j = 0; j < N; j++)
```

In this case, the scope of `j` is limited to the for-loop and cannot be accessed after the loop ends. However, unlike variables declared in the block, its value persists across iterations and is incremented with each pass through the loop. You can also declare the loop index outside the loop, as below:

```

int j; // declare j before loop
for (j = 0; j < N; j++ )
{
    double y = j*h;
    std::cout << "j = " << j << ", y = " << y << std::endl;
}

```

In that case, the index continues to exist after the loop has finished executing.

2.5 C-style arrays

In C++, an *array* is a data structure that stores a collection of values, where each element is accessed using an index, where the index starts from 0. Though C++ offers several ways of defining and working with arrays, here we offer a gentle introduction using what are commonly called *C-style arrays*,² a basic array type inherited from the C language. In Chapter 3, we introduce *C++ vectors*, which are more flexible and will be used throughout the remainder of the text. Students already familiar with C++ vectors are welcome to use them for this chapter's homework instead.

²These are also called “*static arrays*.”

Declaring arrays

File (2.4) demonstrates three ways of declaring C-style arrays:

File 2.4: Three ways of declaring a C-style array.

```
1 // method 1: declare array of 5 elements
2
3 double U[5];
4
5 // method 2: declare and initialize array
6
7 double V[5] = {2, 4, 8, 12, 16};
8
9 // method 3: specify the array size using a const variable
10
11 const int N = 5;
12 double W[N];
```

The differences between these methods are the following:

- Method 1 declares an array `U` of type `double` with 5 elements. The elements are uninitialized and hold indeterminate (“garbage”) values.
- Method 2 declares and initializes the array `V` with explicit values.
- Method 3 declares the array `W`, and specifies the array size using a variable `N`. To use this approach, the variable `N` must be declared before the array, and it must be of type `const int`.

Some basic array operations

Operating on C-style arrays is straightforward, and we leave it to the reader to explore on their own. The minimal example below demonstrates how to access array elements and initialize their values using a for-loop.

```
1 #include <iostream>
2
3 int main()
4 {
5
6     // Declare variables
7     const int N = 7;
8     double delta = 5.0;
9     double h = delta/(N-1);
10
11     // compute grid points y
12     double y[N];
13     for (int j = 0; j < N; j++)
14     {
15         y[j] = j*h;
```

```

16     std::cout << y[j] << std::endl;
17 }
18
19 return 0;
20 }

```

Important characteristics of C-style arrays

When operating on C-style arrays, it is useful to know the following characteristics and limitations:

1. **Fixed size:** The size of a C-style array must be known at compile time and cannot be changed after declaration.
2. **Single data type:** All elements must have the same type (e.g., all `int` or all `double`).
3. **Zero-based indexing:** Array indices start at 0.
4. **No bounds checking:** The compiler does not verify whether an index is valid. Accessing outside the array bounds can produce undefined behavior or crash the program.

Note: A practical limitation of C-style arrays is that their size must be known at compile time. This means you cannot declare a C-style array of size `N`, where `N` is an integer read from the terminal. Using C-style arrays for this chapter's homework requires you to set `N` within the source code. Furthermore, to run the C++ program for multiple `N` values requires you to edit and recompile the source file. Chapter 3 introduces a more flexible array-like data structure, called a *vector*, that does not require the size to be known at compile time.

2.6 The `cmath` header

The `cmath` header provides a collection of common mathematical functions, including exponential, logarithmic, trigonometric, and hyperbolic functions. File (2.5) demonstrates its use to compute $\exp[\sin(2.3\pi)]$.

File 2.5: Using the `cmath` header.

```

1 #include <iostream>
2 #include <cmath>
3
4 int main()
5 {
6     const double pi = 3.141592653589793;
7     double f = std::exp( std::sin( 2.3*pi ) );
8     std::cout << "f = " << f << std::endl;
9
10    return 0;
11 }

```


Several points in File (2.5) deserve discussion:

1. Line 2 includes the `cmath` header.
2. Line 6 declares `pi` as a `const double`. To maintain double precision, 16 significant figures are used. Alternatively, `pi` can be computed using a trigonometric identity:

```
const double pi = 4.0*std::atan(1.0);
```

3. Line 7 uses `std::exp` and `std::sin` to compute `f`.

Compiling and running File (2.5) produces the following output to the terminal:

```
f = 2.2457
```

Note that though the computation is performed in double precision, `std::cout` only displays six significant digits by default. The output precision of `std::cout` can be increased using `std::setprecision`, which is available through the `iomanip` header. For example, to display `f` with 15 significant digits, add `#include <iomanip>` at the top of the file, and use the following statement before printing:

```
std::cout << std::setprecision(15);  
std::cout << "f = " << f << std::endl;
```

This produces the output:

```
f = 2.24569936620199
```

2.7 Homework

Problem 1

Define the physical parameters `Uinf`, `nu`, and `x`, and the integer `N`. Use these to compute the boundary layer thickness `delta` and the grid spacing `h`. For this assignment, use the values $U_\infty = 1$ m/s, $\nu = 1.5 \times 10^{-5}$ m²/s (air at room temperature), and $x = 0.5$ m. You will vary `N` throughout this assignment. Because `N` is used to size C-style arrays, it must be declared directly in the source code as a `const int` rather than read from the terminal with `std::cin` (recall the note in Section 2.5). This means that testing a new value of `N` requires editing `N` in the source file and recompiling the program. Note that `N` must be at least 3 to accommodate the one-sided finite-difference formulas used at y_0 and y_{N-1} .

Problem 2

Declare two C-style arrays of length `N` to store the grid points y_j and the velocity values u_j . Use a `for`-loop to compute and store y_j and u_j .

Problem 3

Declare a C-style array of length N to store the numerically approximated vorticity ω_j . Use a **for**-loop to approximate $\partial u/\partial y$ at every grid point using the finite-difference formulas given in Section 2.1, and compute the vorticity $\omega_j \approx -\partial u_j/\partial y$. Your implementation must correctly apply the centered difference formula at interior points, and the one-sided formulas at $j = 0$ and $j = N - 1$. List the grid points y_j and the corresponding numerical vorticity values ω_j to the terminal.

Problem 4

Declare a C-style array to store the exact vorticity,

$$\omega_{\text{ex}}(y) = -\frac{\partial u}{\partial y} = -\frac{\pi U_\infty}{2\delta} \cos\left(\frac{\pi y}{2\delta}\right), \quad (2.12)$$

and compute it at each grid point using a **for**-loop. Then, compute and display the global relative error,

$$E_\infty(N) = \frac{\|\omega_N - \omega_{\text{ex}}\|_\infty}{\|\omega_{\text{ex}}\|_\infty}, \quad (2.13)$$

where ω_N and ω_{ex} are vectors containing the N numerical and exact values, respectively, of ω_j .

Problem 5

To verify your code, generate a log-log plot of E_∞ versus N . To generate the plots, you may use external software such as MATLAB or Python. Find a best fit to the data in terms of a power law:

$$E_\infty = aN^b, \quad (2.14)$$

and include the best fit as a dashed line in your error plot. Confirm that the observed convergence rate matches the expected order of accuracy of the finite-difference formulas used. Using the best-fit power law, estimate the smallest integer N -value (N^*) for which your code produces a 0.1% relative error ($E_\infty = 0.001$).

Problem 6

Using the N -value N^* determined above, generate a plot of the computed profile of $\omega_j(y_j)$. Plot ω_j on the horizontal-axis and y_j on the vertical axis. Include units in your axes labels, using mm for distances and seconds for time. Interpret the plot physically in terms of

1. the sign of ω ,
2. the location where the magnitude $|\omega|$ is maximized,
3. the location where $|\omega|$ is minimized,
4. Does the vorticity profile have the expected shape? Particularly, discuss how ω_j behaves as it approaches $y_{N-1} = \delta(x)$. Comment on whether the behavior is physical.